

**YENEPOYA INSTITUTE OF ARTS,
SCIENCE,
COMMERCE AND MANAGEMENT**

Project –Low Level Design

on

**Visual Threat Hunting for Manufacturing
Networks**

Student Name: Ahmed Jasim Abdullah

Industry Mentor:
Mr.Sumith
Kumar Shukla

Guided
by:Mr
Vikyath
Shetty

Table of Contents

1. Introduction
2. Detailed Module Design
3. Algorithm Design
4. Data Structures
5. Function-Level Design
6. Error Handling
7. Interface Design
8. Performance Considerations
9. References

1. Introduction

1.1 Scope of Document

This document describes the detailed internal design of the Manufacturing Threat Hunter system, including module architecture, detection algorithms, data structures, function specifications, error handling strategies, and interface design.

1.2 Intended Audience

- Developers
- Evaluators
- Project Reviewers
- Industrial Network Security Analysts

1.3 System Overview

Manufacturing Threat Hunter is an AI-powered cybersecurity dashboard for industrial manufacturing networks. It monitors network traffic logs to detect and visualize threats using an ensemble of four Machine Learning models (Isolation Forest, Local Outlier Factor, K-Means, Random Forest) combined with a Heuristic Rule Engine. The Streamlit-based platform supports live traffic simulation, predictive threat forecasting, and automated PDF/CSV reporting.

2. Detailed Module Design

2.1 Modules

1. Main Application (app.py)

- Streamlit entry point handling UI layout, sidebar controls, and tab rendering
- Manages CSV file upload and delegates to preprocessing and detection pipelines
- Controls Live Simulation Mode loop generating and evaluating synthetic traffic every few seconds
- Displays top KPI metrics: Total Logs, Suspicious Events, Critical Threats, Peak Traffic Time

2. Data Preprocessing Module (utils/preprocessing.py)

- Validates required columns: timestamp, device_ip, protocol, bytes_sent, bytes_received, connections
- Imputes missing numeric values with column medians; fills missing status fields with 'Normal'
- Engineers features: hour and day_of_week from timestamp, total_bytes, protocol_encoded (TCP=0, UDP=1, etc.)
- Caps outliers at the 99th percentile to prevent model skewing
- Can generate synthetic network datasets for testing when no CSV is uploaded

3. Threat Detection Engine (utils/detector.py)

- Contains the ThreatDetector class — the core intelligence of the system
- Trains and runs 4 ML models: Isolation Forest, LOF, K-Means, Random Forest
- Applies 4 heuristic rules to flag known attack patterns
- Calculates a composite Threat Score (0-100) per log entry
- Performs next-hour traffic forecasting using Linear Regression
- Serializes trained models to models/ directory as .pkl files via Joblib

4. Visualization Module (utils/visualizer.py)

- Generates 7 Matplotlib/Seaborn security charts in dark theme
- Charts: Traffic Over Time, Spike Detection, Hour-Day Heatmap, Bytes Scatter Plot, Daily Attack Trend, Protocol Distribution Pie, Anomaly Clusters Scatter

5. Reporting Module (utils/report.py)

- Generates downloadable PDF security reports using FPDF library
- PDF includes: total logs, suspicious events, peak time summary, and top 20 critical threats table
- Exports fully processed dataset as CSV (with ML flags and threat scores) for SIEM integration

3. Algorithm Design

3.1 End-to-End Detection Algorithm

1. Load and validate CSV network log file
2. Impute missing values; engineer features (hour, day_of_week, total_bytes, protocol_encoded)
3. Cap extreme values at 99th percentile
4. Scale features using StandardScaler
5. Train Isolation Forest (contamination=0.10) on feature matrix
6. Train Local Outlier Factor; flag low-density deviations

7. Train K-Means; flag points whose distance to centroid exceeds 90th percentile
8. Apply heuristic rules to generate pseudo-labels for Random Forest training
9. Train Random Forest on pseudo-labeled data for supervised anomaly classification
10. Compute composite Threat Score per record (Volume + Connection + ML + Rule factors)
11. Assign Threat Level: Critical (75+), High (50-74), Medium (25-49), Low (0-24)
12. Run Linear Regression on hourly aggregated bytes_sent to forecast next-hour traffic
13. Flag 'SPIKE LIKELY' if forecast exceeds 95th percentile of historical traffic
14. Render dashboard charts, metrics, and enable PDF/CSV report download

3.2 Heuristic Rule Engine

Rule Name	Condition	Severity
Possible DDoS	bytes_sent > 10,000 AND connections > 100	Critical
Data Exfiltration	bytes_sent > 5,000 AND bytes_received < 500	Critical
Brute Force / Connection Burst	connections > 100	High
Suspicious Spike	bytes_sent > 8,000	Medium

3.3 Threat Scoring Formula

Factor	Max Points	Calculation Method
Volume Factor	30	Normalized: (bytes_sent / max_bytes_sent) x 30
Connection Factor	20	Normalized: (connections / max_connections) x 20
ML Anomaly Factor	30	Flat 30 points if any of the 4 ML models flags the record
Rule Factor	40	40 pts for Critical rules (DDoS/Exfiltration); 25 pts for High/Medium rules

Final Score = Volume + Connection + ML + Rule factors, capped at 100.

3.4 Machine Learning Models

Model	Type	Algorithm	Anomaly Trigger
Isolation Forest	Unsupervised	Random partitioning tree isolation	Contamination threshold = 10%
Local Outlier Factor	Unsupervised	Local density deviation from neighbors	LOF score significantly below neighbors
K-Means Clustering	Clustering	Euclidean distance to cluster centroid	Distance > 90th percentile of all distances
Random Forest	Supervised (pseudo-labeled)	Ensemble decision trees on rule-generated labels	Predicted class = anomaly

3.5 Predictive Forecasting Algorithm

- Aggregates bytes_sent by hour to create an hourly traffic time series
- Trains a Linear Regression model (Scikit-Learn) on the hour index vs. aggregated traffic volume
- Predicts bytes_sent for the next hour (current_hour + 1)
- Compares prediction against the 95th percentile of historical hourly traffic
- Triggers 'SPIKE LIKELY' warning if forecast exceeds the 95th percentile threshold

4. Data Structures

Data Entity	Structure	Description
Raw Network Log	Pandas DataFrame	CSV-loaded traffic records with columns: timestamp, device_ip, protocol, bytes_sent, bytes_received, connections, status
Engineered Feature Set	Pandas DataFrame	Raw data extended with: hour, day_of_week, total_bytes, protocol_encoded, outlier-capped values
Scaled Feature Matrix	NumPy ndarray	StandardScaler-transformed numeric features fed to ML models: bytes_sent, bytes_received, connections, total_bytes, hour, protocol_encoded
Threat Results	Pandas DataFrame	Full dataset annotated with: if_anomaly, lof_anomaly, km_anomaly, rf_anomaly, rule_triggered, threat_score, threat_level
Hourly Traffic Series	Pandas Series	Aggregated bytes_sent grouped by hour; input to Linear Regression forecaster
Rule Labels	Pandas Series (0/1)	Pseudo-labels generated by heuristic rules; used as training targets for Random Forest
Trained ML Models	Joblib .pkl files	Serialized models stored in models/: isolation_forest.pkl, lof.pkl, kmeans.pkl, random_forest.pkl, scaler.pkl
PDF Report	FPDF object / binary	Generated report summarizing stats and top 20 critical threats; downloadable via Streamlit
CSV Export	UTF-8 string	Encoded full processed DataFrame including all anomaly flags and threat scores

5. Function-Level Design

Module	Function	Input	Output	Description
preprocessing.py	load_and_clean(source)	File path or UploadedFile	Cleaned DataFrame	Reads CSV, imputes missing values, normalizes string fields
preprocessing.py	engineer_features(df)	Cleaned DataFrame	Feature DataFrame	Derives hour, day_of_week, total_bytes, protocol_encode
preprocessing.py	cap_outliers(df, cols)	DataFrame, column list	Capped DataFrame	Clips values to 99th percentile per specified column
preprocessing.py	generate_synthetic_data(n)	Integer (row count)	Synthetic DataFrame	Creates fake network log data with injected anomalies for testing
detector.py	ThreatDetector.train(df)	Feature DataFrame	Trained ThreatDetector	Fits all 4 ML models; saves .pkl files; generates pseudo-labels for RF
detector.py	ThreatDetector.predict(df)	Feature DataFrame	Annotated DataFrame	Runs all 4 models + heuristic rules; appends anomaly flags per row
detector.py	ThreatDetector.score_threats(df)	Annotated DataFrame	Scored DataFrame	Computes composite Threat

Module	Function	Input	Output	Description
detector.py	ThreatDetector.forecast_next_hour(df)	Feature DataFrame	Float, bool	Score (0-100) and assigns Threat Level Predicts next-hour bytes_sent returns value and SPIKE_LIKELY flag
detector.py	apply_heuristic_rules(df)	Feature DataFrame	DataFrame with rule_triggered column	Applies 4 attack pattern rules; records rule name per flagged row
visualizer.py	plot_traffic_over_time(df)	DataFrame	Matplotlib Figure	Line chart of bytes_sent over timestamp
visualizer.py	plot_spike_detection(df)	DataFrame	Matplotlib Figure	Highlights records exceeding spike threshold
visualizer.py	plot_heatmap(df)	DataFrame	Seaborn Figure	2D heatmap: suspicious events by hour vs. day_of_week
visualizer.py	plot_scatter(df)	DataFrame	Matplotlib Figure	Scatter: bytes_sent vs bytes_received, colored by connections
visualizer.py	plot_daily_attack_trend(df)	DataFrame	Matplotlib Figure	Bar chart of suspicious events

Module	Function	Input	Output	Description
				counts per calendar day
visualizer.py	plot_protocol_distribution(df)	DataFrame	Matplotlib Figure	Pie chart of traffic volume by protocol
visualizer.py	plot_anomaly_clusters(df)	DataFrame	Matplotlib Figure	Scatter of records colored by Threat Score
report.py	generate_pdf(df, stats)	DataFrame, dict	FPDF bytes	Creates PDF with summary stats and top 20 critical threats table
report.py	export_csv(df)	DataFrame	UTF-8 string	Encodes full processed DataFrame as CSV string for download

6. Error Handling

Error Condition	Module	Handling Strategy
Invalid or corrupted CSV file	preprocessing.py	Try/except around pd.read_csv(); Streamlit error message displayed; system loads synthetic data as fallback
Missing required schema columns	preprocessing.py	Column check raises ValueError listing missing fields; user prompted to correct file
All-null column after imputation	preprocessing.py	Column dropped from feature set; warning logged; remaining features used for ML

Error Condition	Module	Handling Strategy
Insufficient data for K-Means	detector.py	Minimum row count check before fitting; K-Means skipped and flagged as unavailable if below threshold
Random Forest training failure (all same pseudo-label)	detector.py	If pseudo-labels are entirely one class, RF training is skipped; other 3 models continue
Forecast fails due to insufficient hourly data	detector.py	Linear Regression requires ≥ 2 data points; if fewer, forecast returns None and UI hides spike warning
PDF generation error	report.py	Try/except around FPDF operations; error message shown; CSV export remains available
Live simulation crash	app.py	Simulation loop wrapped in try/except; on error, loop stops gracefully and user notified
Model .pkl file missing or corrupt	detector.py	Joblib load wrapped in try/except; system retrains from scratch on current data if file fails to load

7. Interface Design

7.1 User Interface

- Web-based dashboard built with Streamlit, custom dark-mode CSS theme for SOC/industrial aesthetic
- Sidebar: CSV file upload widget, 'Run Analysis' button, Live Mode toggle, PDF/CSV download buttons
- Top of main panel: 4 KPI metric cards — Total Logs, Suspicious Events, Critical Threats, Peak Traffic Time

7.2 Tabbed Output Interface

Tab	Contents
Security Analytics	7 Matplotlib/Seaborn charts: Traffic Over Time, Spike Detection, Hour-Day Heatmap, Bytes Scatter, Daily Attack Trend, Protocol Pie, Anomaly Clusters
Detailed Data & AI Insights	Per-model anomaly counts (IF, LOF, K-Means, RF); next-hour traffic forecast with SPIKE LIKELY warning; raw processed data table with all flags and threat scores

7.3 Live Simulation Mode

- Toggle in sidebar activates continuous simulation loop
- Every few seconds: generates 1 new synthetic log entry (10% chance of injected DDoS/spike anomaly)
- New entry processed through full detection pipeline in real time
- Dashboard KPI metrics and charts update dynamically without full page reload

7.4 Interface Screenshots

The following screenshots should be inserted here:

- Screenshot 1: Sidebar with CSV upload, Run Analysis button, Live Mode toggle
- Screenshot 2: Top KPI metric cards (Total Logs, Suspicious Events, Critical Threats, Peak Traffic)
- Screenshot 3: Security Analytics tab showing all 7 charts
- Screenshot 4: Detailed Data & AI Insights tab with model breakdown and forecast
- Screenshot 5: Sample PDF report output

8. Performance Considerations

- All data processing is in-memory using Pandas and NumPy — no database overhead
- Four ML models are trained once per analysis run; serialized to .pkl via Joblib to avoid retraining on re-runs
- StandardScaler is also serialized alongside models to ensure consistent feature scaling during inference
- Outlier capping (99th percentile) prevents extreme values from inflating model training time
- Linear Regression forecaster operates on aggregated hourly data (small dataset), ensuring near-instant predictions

- Live simulation generates one record at a time, keeping real-time inference lightweight
- Matplotlib/Seaborn charts generated server-side and rendered as static images, keeping client-side load minimal
- PDF generation via FPDF is synchronous; for very large datasets, it is run only on the top 20 critical threats to bound execution time
- Synthetic data generator allows performance benchmarking without needing real network captures

9. References

- Streamlit Documentation — <https://docs.streamlit.io>
- Pandas Documentation — <https://pandas.pydata.org/docs>
- NumPy Documentation — <https://numpy.org/doc>
- Scikit-Learn Documentation (IsolationForest, LOF, KMeans, RandomForest, LinearRegression) — <https://scikit-learn.org/stable>
- Matplotlib Documentation — <https://matplotlib.org/stable/contents.html>
- Seaborn Documentation — <https://seaborn.pydata.org>
- FPDF2 Documentation — <https://py-pdf.github.io/fpdf2>
- Joblib Documentation — <https://joblib.readthedocs.io>
- Python Documentation — <https://docs.python.org>